ASSIGNMENT 1 — SOFTWARE DEVELOPMENT PLAN (SDP)

F-order: Dormitory Football Reservation Application

Deliverable: F-order_D1_SDP.pdf | **Due:** Week 4 — 22 October 2026

Team Members: Okan Bölük (Team Captain), Yahya Yahşi, Kateryna Shapilova

1. Problem and Project Context

- **What problem are you solving?** Football pitch reservations and player organization in our student dormitory are currently managed without any formal structure through an unorganized WhatsApp group.
- **Who experiences the problem?** University students residing in the dormitory who want to play football regularly, as well as the dormitory administration responsible for maintaining pitch order.
- **What evidence or observation supports the problem?** High message volume in the WhatsApp group causes match announcements to be overlooked, verbal bookings frequently overlap, last-minute no-shows leave teams short-handed or force match cancellations, and finding substitute players urgently creates severe communication chaos.
- **What is the intended outcome for users or an organization?** An automated mobile application where students can view hourly pitch availability and book individual slots, a strict capacity of 14 regular and 2 substitute players is automatically managed, registered match players can communicate via a temporary real-time chat portal, and no-show players are penalized through a peer-reporting ban mechanism.

2. Stakeholders and Users

Stakeholder / User Group	Goals	Constraints	Possible Conflicts
1. Regular & Substitute Players (Dormitory Students)	Quickly find available match hours, register easily, start matches with a full 14-player squad, and rapidly fill missing spots using substitutes.	Mandatory rule to arrive at the pitch at least 10 minutes before kickoff; waiting in the 2-player substitute queue when the regular list is full.	Grievances when a regular player fails to show up at the last minute; substitute players being called too late or not getting a chance to play.
2. System Administrators / Dorm Management	Ensure fair, orderly, and conflict-free utilization of the dormitory football pitch by penalizing rule-breakers (no-shows).	Inability to station physical staff at the pitch 24/7; zero budget allocation for dedicated server infrastructure or paid maintenance.	Disputes from students claiming they were unfairly flagged as a "no-show" by peers; accounts created with fake room or phone details.

3. Scope

In Scope for the First Release

- User registration and authentication requiring full name, phone number, dormitory block, and room number.
- Interactive daily/hourly block calendar and individual match slot booking engine.
- Automated capacity control for **14 Regular + 2 Substitute** players per hourly slot, with automatic promotion of substitutes when a regular player cancels.
- Visibility of registered players' contact numbers for pre-match coordination and a persistent "arrive 10 minutes early" rule reminder.
- A **temporary real-time chat portal** accessible exclusively to players registered for that specific match hour, automatically wiped when the match ends.
- Peer-to-Peer (P2P) no-show reporting during the match window and an automated temporary booking ban for users exceeding the report threshold.

Explicitly Out of Scope

- Team-based bulk bookings, team captaincy selection, and in-app friend lists or social networking features.
- Online payment processing, deposit collection, or pitch fee-splitting features.
- Hardware-based physical attendance verification at the pitch (e.g., QR turnstiles, GPS geofencing, or Bluetooth beacons).

Initial Success Criteria

1. Complete replacement of the dormitory WhatsApp group for match scheduling, migrating all reservation traffic to F-order.
2. At least **90% of weekly scheduled matches** kicking off with a full 14-player squad.
3. A measurable decrease in player no-show rates within the first month of deployment due to the automated ban system.
4. Zero double-booking errors when multiple students simultaneously tap the last remaining regular or substitute slot.

Major Assumptions and Dependencies

- **Assumptions:** Dormitory students possess smartphones with internet access, provide accurate room and phone information during registration, and use the peer-reporting mechanism fairly (enforced via multi-player consensus thresholds).
- **Dependencies:** The application depends on the **Supabase** free-tier cloud infrastructure (PostgreSQL Database, Auth, and Realtime Broadcast) and the cross-platform **Flutter** SDK.

4. Initial Process and Work Plan

Process Model: To allow all three team members to develop in parallel without blocking one another, the team uses an Agile **Feature-Based (Vertical Slicing / Full-Stack Domain Ownership)** workflow. Each member owns their module end-to-end from the Flutter UI down to the database logic.

Team Member & Role	Domain Module	Core Responsibilities
Okan Bölük (Team Captain)	Core Booking Engine	Main calendar UI, hourly availability blocks, 14 regular + 2 substitute booking logic, concurrency/race-condition state management, and automatic substitute promotion.
Yahya Yahşi	Real-Time Communications	Temporary match chat portal, WebSocket / Supabase Realtime Broadcast channels, in-memory session handling, and automatic message destruction after match completion.
Kateryna Shapilova	Identity, Trust & Ban System	Initial registration (phone, block, room number), user authorization, peer-to-peer no-show reporting verification algorithm, and automated temporary ban enforcement.

Initial Milestones Through Week 12

- **Week 4:** Submission of the Software Development Plan (SDP), GitHub repository initialization, and role finalization.
- **Weeks 5–6:** Completion of the Software Requirements Specification (SRS), PostgreSQL database schema design, and UI wireframes.
- **Weeks 7–8:** Implementation of User Registration/Auth and the Core Booking Engine prototype (hourly blocks and 14+2 capacity rules).
- **Weeks 9–10:** Integration of the Temporary Match Chat Portal and the Peer-to-Peer No-Show Reporting & Ban algorithm.
- **Week 11:** End-to-end integration testing, race-condition stress testing, and bug fixing.
- **Week 12:** Release v1.0 live demonstration and final project delivery.

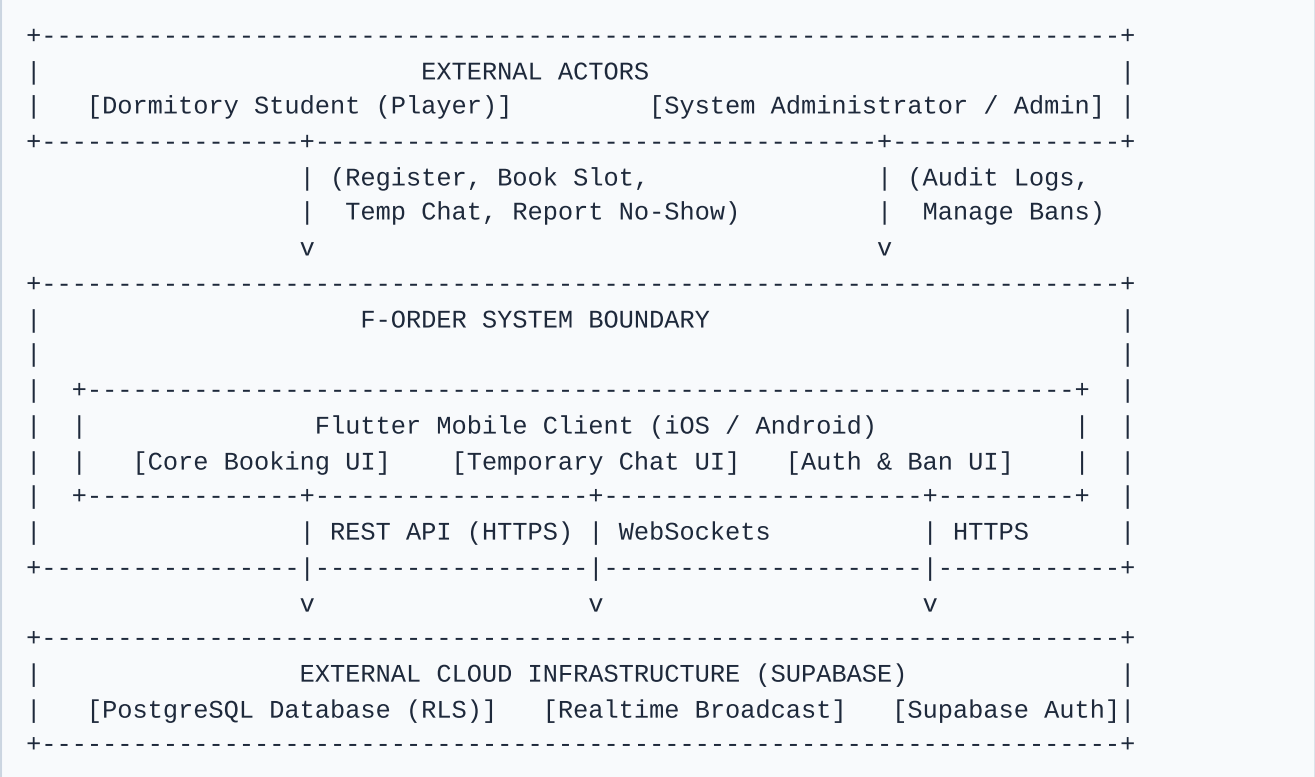
Communication, Tracking, and Change Management

- **Communication & Decision-Making:** Daily asynchronous updates occur via messaging, supported by weekly sync meetings. Architectural decisions are made by consensus; technical deadlocks are resolved by a 2/3 majority vote led by the Team Captain.
- **Work Tracking:** All tasks and user stories are tracked on a **GitHub Projects Kanban Board** (*To Do, In Progress, In Review, Done*) linked to GitHub Issues.
- **Handling Changes:** Any change to scope or technical direction is evaluated during weekly team meetings and logged with its justification in the SDP/SRS revision history.

5. Technology and Architecture Direction

Layer / Component	Technology Choice	Reason for Choice	Trade-offs
Mobile Client (Frontend)	Flutter (Dart) & Riverpod/Provider	Single codebase for iOS and Android; rich widget set enables rapid development of hourly calendar blocks and chat interfaces.	Slightly larger initial binary size compared to native apps, accepted in exchange for development speed.
Backend & Database (BaaS)	Supabase (PostgreSQL & Auth)	Zero server maintenance and free-tier hosting; relational PostgreSQL with row-level locking prevents concurrent double bookings.	Free-tier imposes storage and connection limits, requiring strict avoidance of unnecessary data persistence.
Real-Time Messaging	Supabase Realtime (WebSockets)	Broadcast feature transmits match chat messages directly in memory between connected clients without writing to disk.	Messages are ephemeral and disappear if a user closes the app, which aligns with the temporary match chat design.

Initial System-Boundary / Context Diagram



Key Quality Attributes

1. **Reliability & Concurrency:** Atomic PostgreSQL transactions and row-level locking guarantee that simultaneous booking requests for the 14th regular slot or 2nd substitute slot never exceed capacity.
2. **Performance:** Calendar slot availability updates and temporary chat messages propagate across connected clients with sub-second latency.
3. **Usability:** Students can inspect hourly blocks, join a match, or report an absent player in no more than two taps from the main dashboard.

6. User Experience or Interaction Evidence

The core user interaction flow follows four streamlined screens: **Registration Screen** (Name, Phone, Block, Room) → **Main Dashboard & Hourly Calendar Blocks** (showing 14/14 full or open slots and the 10-minute rule banner) → **Your Next Match & Temporary Chat Room** (displaying regular/substitute counts and launching the ephemeral chat) → **Report No-Show Action** (flagging absent players to trigger the ban algorithm).



Figure 1: F-order Mobile Application Dashboard Wireframe showing user block/room info, the 10-minute early rule notice, hourly booking blocks, next match status (14 regular / 2 substitute), temporary chat entry, and no-show reporting.

7. Repository and Collaboration Setup

- **GitHub Repository URL:** `https://github.com/Okanboluk/f-order`

- **Branch and Pull-Request Workflow:**
  - `main`: Protected production branch containing only tested, release-ready code.
  - `develop`: Primary integration branch where completed feature modules are merged.
  - `feature/<module-name>`: Isolated working branches for each domain (`feature/core-booking`, `feature/realtime-chat`, `feature/identity-ban`).
  - **PR Rule:** Direct pushes to `main` and `develop` are disabled. Every feature requires a Pull Request into `develop` and approval (Code Review) from at least one other team member before merging.

Initial README

```
# F-order — Dormitory Football Reservation Application

## Overview
F-order replaces chaotic dormitory WhatsApp groups with a structured mobile booking
platform. It provides hourly pitch reservations, automated 14 regular + 2
substitute squad management, ephemeral match chat rooms, and a peer-verified no-
show ban system.

## Team & Domain Ownership
- Okan Bölük (Team Captain): Core Booking Engine (Calendar UI, State & Concurrency)
- Yahya Yahşi: Real-Time Communications (Supabase Broadcast Ephemeral Chat)
- Kateryna Shapilova: Identity, Trust & Ban System (Auth, P2P No-Show Verification)

## Tech Stack
- Frontend: Flutter (iOS & Android)
- Backend: Supabase (PostgreSQL, Auth, Realtime WebSockets)

## Quick Start
1. Clone repository: `git clone https://github.com/0kanboluk/f-order.git`
2. Install packages: `flutter pub get`
3. Configure `.env` with `SUPABASE_URL` and `SUPABASE_ANON_KEY`
4. Run application: `flutter run`
```

8. Initial Risk Register

Risk	Likelihood	Impact	Early Warning Signal	Mitigation	Contingency
1. Temporary chat overloading database storage	Medium	High	Rapid weekly growth in Supabase database storage metrics.	Transmit chat messages via Supabase Realtime Broadcast in RAM without writing rows to disk.	Run an automated post-match cleanup trigger to purge any residual session data immediately.

Risk	Likelihood	Impact	Early Warning Signal	Mitigation	Contingency
2. Inability to physically verify the "10-Min Early" rule	High	Medium	Players arriving late at kickoff and matches starting behind schedule.	Display a persistent top banner reminder and enable Peer-to-Peer "Report No-Show" flagging for active players.	Call/notify the missing player via the app roster, promote a substitute player, and issue an automated ban.
3. Fake registrations (invalid phone or room numbers)	Medium	High	Unreachable phone numbers or non-existent dorm rooms appearing in match rosters.	Enforce strict phone/room format validation at sign-up and require initial admin verification.	Purge fraudulent accounts from PostgreSQL and blacklist the associated phone/device identifier.
4. Race condition on the last available booking slot	High	High	More than 14 regular players appearing in a single hourly slot.	Execute slot reservations inside an atomic PostgreSQL RPC function using row-level locking.	Automatically place the later timestamp into the 2-player substitute queue and notify the user.
5. Abuse of the P2P No-Show reporting button	Low	High	Appeals from present players claiming they were falsely banned by peers.	Require a consensus threshold (e.g., at least 4 active match players flagging the same user) before triggering a ban.	Allow administrators to manually revoke unjust bans and penalize users who submit false reports.

9. Responsible AI-Use Plan

- **Permitted Activities:** AI-assisted tools may be used for generating standard Flutter UI boilerplate, form validation patterns, debugging PostgreSQL queries, and drafting initial documentation structure.
- **Review and Testing Process:** No AI-generated code or text is committed directly to `develop` or `main`. The module owner must test all code locally and submit it via Pull Request for line-by-line review by a teammate. Every team member must fully understand and be able to explain all submitted code and documentation.
- **Dependencies, Licenses, and Sensitive Data Checks:** Personal student data (real phone numbers, room numbers) and secret keys (`.env` credentials) will never be entered into AI prompts. Any third-party package suggested by AI is verified on `pub.dev` for permissive open-source licensing (MIT, BSD, Apache 2.0) before adoption.
- **Documentation of AI-Assisted Decisions:** Significant architectural or algorithmic decisions shaped with AI assistance are explicitly noted in Pull Request descriptions and project revision logs.

Submission Checklist

☑ PDF follows the required filename (`F-order_D1_SDP.pdf`).

☑ Repository and project board are accessible to the instructor.

☑ Scope includes explicit exclusions.

☑ Technology choices include reasons and trade-offs.

☑ At least one initial diagram is included.

☑ Risks include concrete mitigations.

☑ All team members contributed and can explain the submission.